DÉDICACE

Je voudrais remercier Dieu pour toute l'énergie qu'il m'a accordé durant ma formation.

Je voudrais dédier mon travail à toutes les personnes qui m'ont aidé durant ma formation

Je voudrais remercier principalement ma mère, mes chers frère et sœurs ainsi que toute ma famille sans qui je n'aurais jamais pu faire ce travail.

REMERCIEMENTS

Mes sincères remerciements vont à l'endroit de :

- toute la Communauté, de l'EIG en particulier mes formateurs qui m'ont suivi dans le cadre de cette formation en Développement Web, pour la disponibilité et le partage dont ils ont fait preuve ;

- M. Monyévédo Firmin TOVODOUNNNON, Directeur des Études de L'EIG ;

- M. DEGBOE Juste, mon mentor, pour avoir accepté de diriger ce travail ;

- Mme FLEURY Yolande, mon maître de stage, et ses collaborateurs directs M. AHOUNOU Pavel et M. LAINÉ Aurélio pour le suivi, la disponibilité et les nombreux conseils éclairés durant le stage ;

- M. Sénou Clovis ALIHONOU pour son soutient et sa disponibilité

-Mme TÉTÉVI Eyram pour son soutient

-M. Cyrus HESSOU pour son soutient et sa disponibilité

- mes frère et sœurs, pour l'amour et l'affection qu'ils nous portent ;

- enfin, à mes parents et amis pour leur soutien et qui de près ou de loin, ont contribué à la réalisation de ce mémoire ; Trouvez ici l'expression de notre profonde gratitude

PLAN

INTRODUCTION

I-Chapitre1 : **CADRE THÉORIQUE**

1-1 Contexte, Problématique et Objectifs

1-2 Définition de quelques concepts

1-2-1 Définition plateforme de gestion universitaire

1-2-2 Définition gestion des cours

1-2-3 Définition gestion des activités

1-2-4 Définition gestion de paiement

1-2-5 Système de paiement en ligne

II-Chapitre2 : **ANALYSE ET CONCEPTION DU SYSTEME**

2-1 Spécialisation des besoins fonctionnel

2-1-1 Identification des acteurs

2-1-2 Besoins fonctionnels

2-1-3 Besoins non fonctionnels

2-2 Modélisation des besoins fonctionnels

2-2-1 Diagramme de cas d'utilisateur

2-2-2 Diagramme de classe

2-3-3 Modélisation de la base de données

III Chapitre3 : **IMPLEMENTATION DU SYSTEME**

3-1 Technologie utilisé

3-1-1 Bootstrap

3-1-2 JavaScript

3-1-3 Laravel

3-2 Les outils utilisés

3-2-1 PhpMyAdmin

3-2-2 Draw.io Diagram

3-2-3 Xamp

3-3 Architecture de la plateforme

3-4 Aperçu visuel de l'interface utilisateur de la plateforme

3 -5 Implémentation

3-5-1 Installation et organisation

3-5-2 Les routes

3-5-3 Les migrations

3-5-4 Les classes model

3-5-5 Les Contrôleurs

3-5-6 Les vues

Conclusion

INTRODUCTION

Le développement d'une plateforme de gestion des cours, des activités universitaires s'inscrit dans un cadre d'innovation technologique visant à simplifier et automatiser le processus académique. Elle permet une centralisation de toutes les informations relatives au suivi de la gestion des cours et des activités universitaires. En plus de la gestion des cours et activités cette plateforme va intégrer également la fonctionnalité liée au paiement des frais de scolarité, permettant de normaliser cette procédure administrative essentielle. Cette plateforme participera à la modernisation de l'administration universitaire en facilitant la gestion des données. Le secteur éducatif a connu des changements majeurs ces dernière années avec l'introduction du numérique qui pourra rendre efficace la gestion administrative et pédagogique des universités. La gestion efficace des cours, des activités universitaires et de paiement de frais de scolarité est un défi majeur pour les universités. Une bonne organisation de ces éléments est essentielle pour garantir le bon fonctionnement de l'université, le suivi des étudiants et la fluidité administrative. Cependant de nombreuse université continuent d'utiliser des systèmes de gestion non centralisée pour la gestion administrative universitaire. De plus la multiplicité des tâches et la complexité des processus, l'augmentation du nombre d'étudiant entraine des inefficacités, des erreurs administratives. La bonne gestion des cours, des activités universitaires et de paiement de frais de scolarité devient un facteur déterminant dans la satisfaction des étudiants et de l'administrations universitaire. C'est dans ce cadre que nous avons choisir de mener une étude sur le thème : **<<Développement d'une plateforme de gestions des cours, des activités universitaires et de paiement de frais de scolarité.>>** Ainsi cette étude vise à rendre efficace la gestion administrative des universités. Nôtre étude est

structurée en trois chapitres : le premier consacré au cadre institutionnel de l'étude au déroulement de stage. Dans le deuxième chapitre, nous montrerons les cadres théoriques, l'analyse et la conception du système et cela avant d'entamer la partie applicative dans le but de bien définir, clarifier les fonctionnalités importantes de la plateforme. Nous avons pour aspect de définir les besoins fonctionnels et non fonctionnel qui sont considéré comme des fonctionnalité de la plateforme. Dans le troisième nous présenterons la réalisation de notre plateforme comportant l'environnement de développement et nous détailleront ensuite les différentes réalisations de la plateforme.

I -Chapitre1 : **CADRE THÉORIQUE**

1-1 Contexte, Problématique et objectif

-Contexte

Le secteur éducatif a connu des changements majeurs ces dernière années avec l'introduction du numérique visant à rendre efficaces la gestion administrative et pédagogique. La gestion efficace des cours, des activités universitaires et de paiement de frais de scolarité est un défi majeur pour les universités Une bonne organisation de ces éléments est essentielle pour garantir le bon fonctionnement de l'université, le suivi des étudiants et la fluidité administrative. Cependant de nombreuses universités continuent d'utiliser des systèmes de gestion séparer la gestion administrative. De plus la multiplicité des tâches et la complexité des processus peuvent parfois entrainer des inefficacités, des erreurs administratives. La bonne gestion des cours, des activités et de paiement de frais de scolarité devient un facteur déterminant dans la satisfaction des étudiant, des professeurs et de l'administration elle-même.

-Problématique

Les université se trouvent confronté à des difficultés de gestion des cours ,des activités universitaire et de paiement de frais de scolarité ce qui entraine l'inefficacité qui impact la qualité du service qu'offres leur système de gestion universitaire Face à cette difficulté une question est posé : Comment optimiser la gestion des cours ,des activité universitaire et de paiement de frais de scolarité tout en

garantissant l'amélioration de l'expérience utilisateurs tant aux étudiants et à l'administration universitaire ?

- **Objectif**

  L'objectif principal est de centralisé et automatisé la gestion des cours, des activités universitaires et de paiement de frais de scolarité afin d'optimiser l'administration universitaire et améliorer de l'expérience utilisateur tant pour les étudiants et pour l'administration universitaire. Les objectifs spécifiques :

  * Concevoir et mettre en place un système pour faciliter la gestion des cours et activités universitaire.

  * Simplifier le paiement des frais de scolarité : proposer un système de paiement en ligne facilitant la gestion du paiement de scolarité

1-2 Définition de quelques concepts

1-2-1 Plateforme de gestion universitaire : Une plateforme de gestion universitaire est un système d'information intégré qui permet aux établissements d'enseignement supérieur de gérer l'ensemble de leurs processus administratifs, pédagogiques et financiers.

1-2-2 Système de paiement en ligne : Un système de paiement en ligne est une solution technologique qui permet d'effectuer des transactions financières via Internet.

II-Chapitre2 : **Analyse et Conception du système**

Introduction

Avant de commencer à créer l'application, il est important de passer par une étape essentielle **:** la phase de spécification. Elle permet de bien définir et de clarifier les principales fonctionnalités que doit proposer notre application. Ce chapitre a donc pour but d'expliquer clairement les actions que l'application doit permettre (les besoins fonctionnels)**,** les règles à suivre pour assurer son bon fonctionnement (les besoins non fonctionnels)**,** ainsi que les objectifs que l'on souhaite atteindre**.**

2-1 Spécialisation des besoins fonctionnels

L'analyse du sujet ainsi que la compréhension des besoins des utilisateurs, nous a permis de définir les différentes fonctionnalités que notre application va proposer. Ensuite, les contraintes à respecter pour que le système fonctionne correctement seront présentées comme des **besoins non fonctionnels**.

2-1-2 Identification des acteurs

Les acteurs que nous avons réussi à identifier sont :

Étudiant : L'étudiant accède aux cours, suit son parcours académique.

Professeur : Le professeur est responsable de la création et de la gestion des cours.

Administrateur : L'administrateur gère l'ensemble de la plateforme, en contrôlant les utilisateurs, les cours, et les paramètres globaux du système.

2-1-2 Besoins fonctionnels

Dans cette partie, on présente tout ce que notre application web doit pouvoir faire pour bien fonctionner. Les fonctions attendues dépendent du rôle de chaque utilisateur (comme l'étudiant, le professeur ou l'administrateur).

C'est pourquoi nous avons décrit les besoins de chaque utilisateur séparément, en fonction de ce qu'il doit pouvoir faire.

Voici donc, de manière claire, les principales fonctionnalités que notre application doit offrir.

Le système doit permettre

Les utilisateurs doivent s'authentifier avec un identifiant et un mot de passe pour accéder aux différentes fonctionnalités.

Le système doit permettre à l'étudiant

De consulter les cours disponibles

Voit son emploi du temps

Adhérer à un club universitaire

Voit les annonces

Voit les activités

Payer sa scolarité

Le système doit permettre au professeur

Ajouter / modifier/ supprimer ses propres cours

Téléverse des documents de cours

Le système doit permettre à l'administrateur

Gère tous les utilisateurs (professeurs, étudiants…)

Gère les paiements

Gère les annonces universitaires

Gère les activités universitaires

Gère l’emploi du temp

Gère les matières

Gère les unités d’enseignement

2-1-3 Besoins non fonctionnels

Il s'agit des besoins qui caractérisent le système. Ce sont des besoins en matière de performance. Ces besoins peuvent concerner les contraintes d'implémentation comme le langage de programmation, le type du système de gestion de base de données.

Toutes les améliorations qu’on va apporter doivent suivre certains besoins précis.

Expérience utilisateur :

L’application doit être facile à utiliser, les interfaces utilisateurs doivent être conviviales c'est-à-dire simples, ergonomiques et adaptées à l'utilisateur.

Fiabilité : les informations de la plateforme doivent être fiable et sûre.

Disponibilité : La plateforme doit être disponible à tout moment pour être utilisé par les utilisateurs.

Sécurité : La plateforme contient des informations personnelles et importantes, donc elle doit être bien protégée.

La performance : La plateforme doit se charger rapidement et facile à utiliser, pour que les utilisateurs puissent naviguer sans lenteur ni difficulté.

2-2 Modélisation des besoins fonctionnels

Avant de commencer la conception, il est important de bien identifier les besoins fonctionnels pour avoir une vision claire de ce que la plateforme doit faire.

2-2-1 Diagramme de cas d’utilisation

La connaissance des fonctionnalités à implémenter dans la plateforme est important pour la réalisation du diagramme de cas d’utilisation

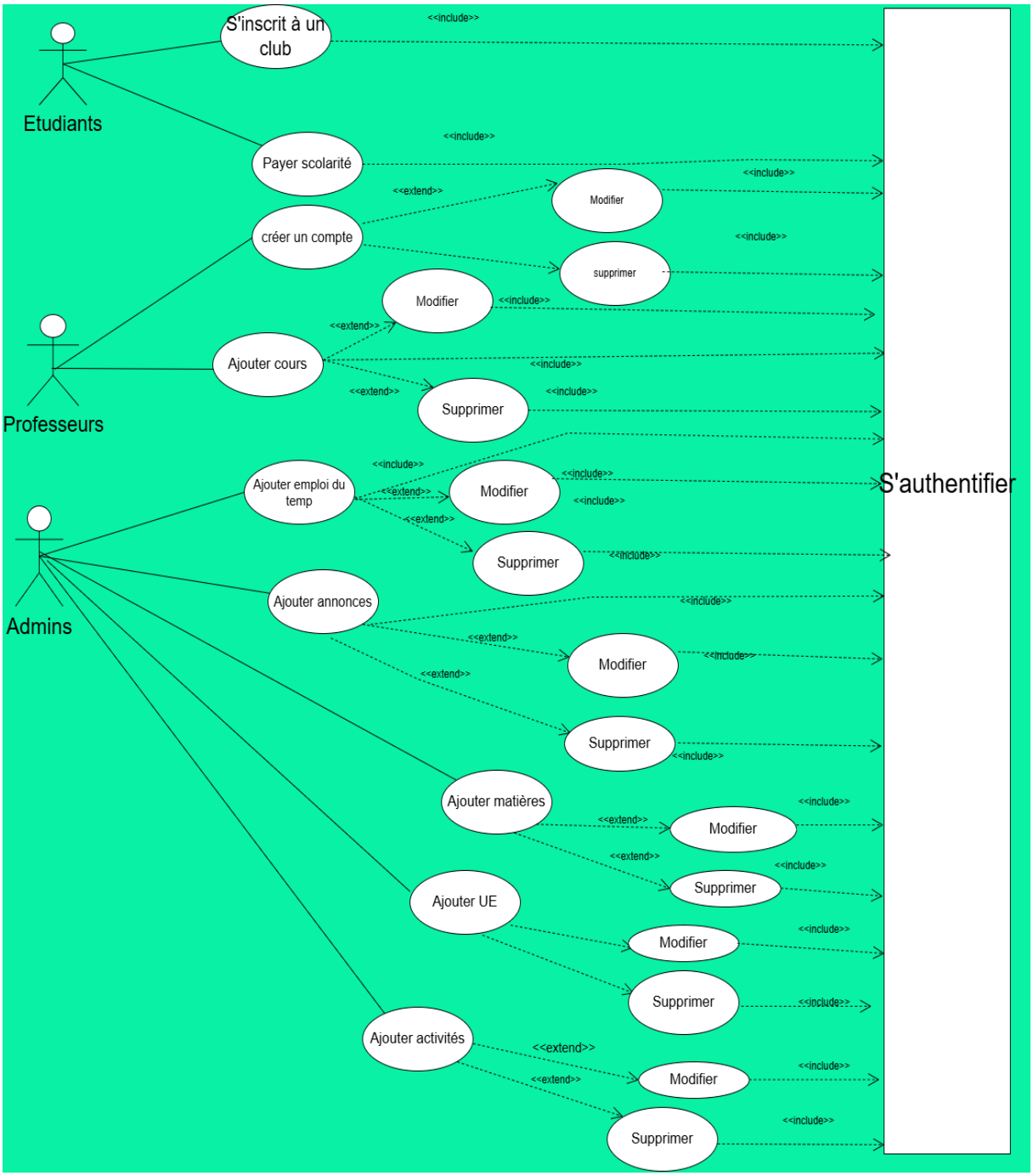

Figure1 : Diagramme de cas d'utilisation de la plateforme de gestion des cours, des activités universitaires et de paiement de scolarité

La figure ci-dessus montre le diagramme de cas d’utilisation principal, qui concerne la gestion des cours, des activités et de paiement de frais de scolarité. On voit que les principaux utilisateurs sont l’étudiant le professeur et l’administrateur. La plateforme donc doit leur permettre à l’étudiant de voir les cours s’inscrit à un club, au professeur d’ajouter une cour avec la possibilité de modifier et supprimer. L’administrateur qui s’occupe de la gestion des emplois du temp des annonces et activités.

2-2-2 Diagramme de classe

La firgure2 est composé des informations suivantes

Le diagramme de classe ci-dessous illustre la structure statique (les classes, les attributs, et les relations) de la plateforme, en représentant les différentes entités ainsi que les relations entre elles. Il permet d’avoir une vision claire de l’organisation des données et de leur interaction au sein du système.

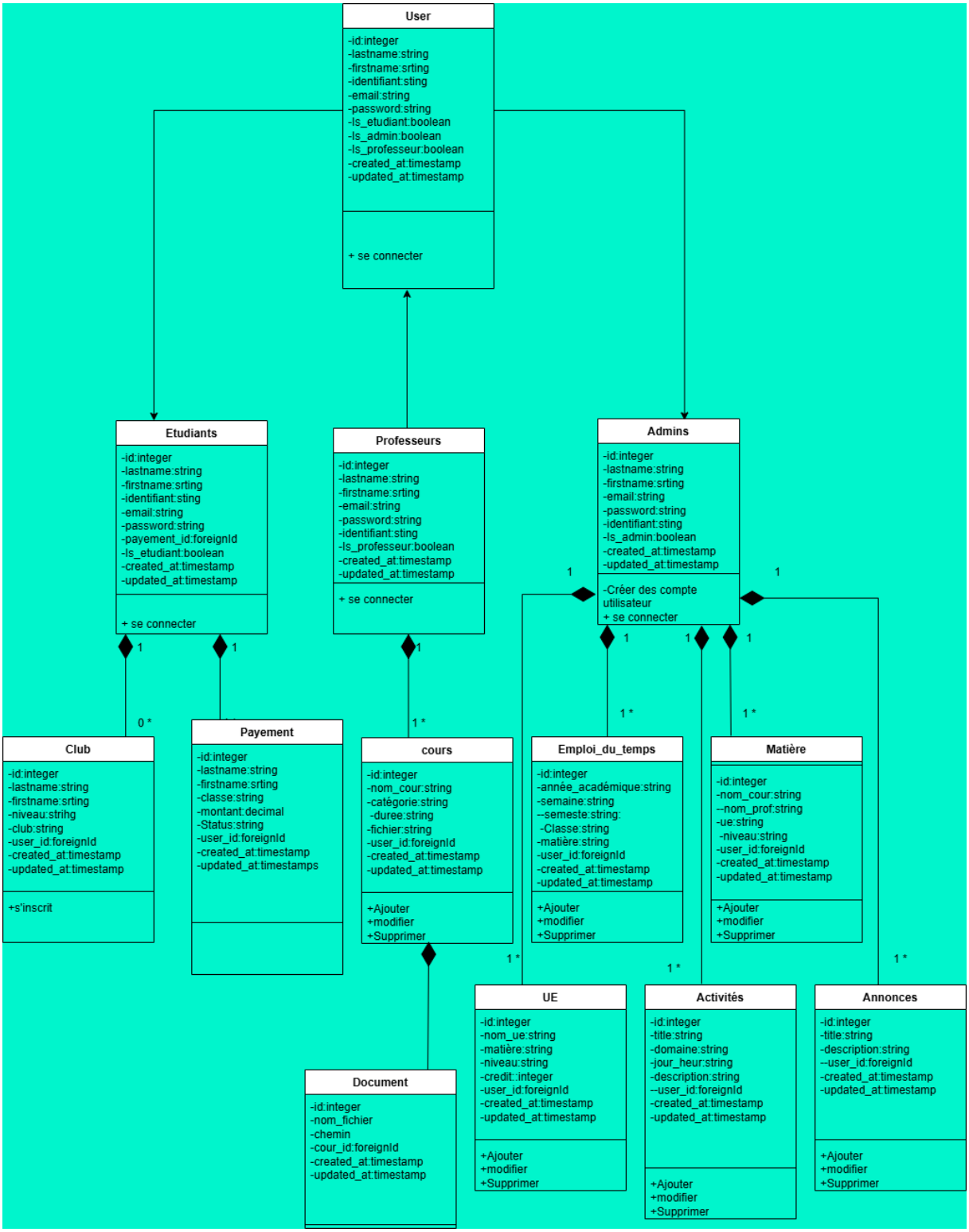

Figure2 : Diagramme de classe de la plateforme

2-2-3 Modélisation de la base de données

La modélisation de la base de données permet de représenter de manière claire et organisée toutes les informations que la plateforme va gérer, ainsi que les relations entre ces informations.

Utilisateur (id, firstname, lastname, identifiant, email, mot_de_passe, Is_admin, Is_professeur, Is_etudtant**)**

Club (id, firstname, lastname, niveau, club, user_id**)**

Cour (Id, nom_cour, categorie, durée, user_id.)

Document (id, nom_fichier, chemin, cours_id)

Paiement (id, firstname, lastname, classe, montant status, montant, user_id**)**

Annonce (id, title, description, user_id**)**

Activité (id, title, domaine, jour_heur, description, user_id)

UE (nom_ue, matiere, niveau, credit, user_id)

Matiere (nom_cour, nom_prof, ue, niveau, user_id)

Emploi du temp (semestre, semaine, classe, matiere, user_id**)**

Au cours de ce chapitre, nous avons détailler notre projet, c'est -à-dire les objectifs majeurs à prendre en compte, nous avons aussi réalisé une analyse profonde en précisant les différentes fonctionnalités du système.

Dans le chapitre suivant, nous allons entamer la phase de l'implémentation

III Chapitre3

IMPLEMENTATION DU SYSTEME

3-1 Technologie utilisée

Ce chapitre représente la dernière partie il traite la phase qui a pour objectif l'implémentation de notre application. Nous débutons, tout d'abord, par la description des technologies utilisées développer notre application. Ensuite nous mentionnons les outils technologiques utilisés. Finalement nous donnons un aperçu sur le travail réalisé en termes de code.

3-1-1 Bootstrap

Bootstrap est une boîte à outils open source pour le développement avec HTML, CSS et JS, propose un prototypage rapide des idées, permet de créer une application entière, un système de grille réactif, de nombreux composants prédéfinis et des plugins puissants construits sur jQuery

3-1-2 JavaScript

JavaScript est un langage de programmation utilisé principalement pour rendre les pages web interactives en manipulant le contenu HTML et CSS, et en réagissant aux actions des utilisateurs. Il permet également de communiquer avec des serveurs de manière asynchrone, sans recharger la page, et peut être utilisé à la fois côté client (navigateur) et côté serveur avec des technologies comme Node.js.

3-1-3 Laravel

Laravel est un Framework PHP o qui facilite le développement d'applications web . Il offre des outils puissants pour la gestion des bases de données, l'authentification des utilisateurs, la gestion des routes, la validation des données, et bien plus, tout en suivant le principe de conception MVC (Modèle-Vue-Contrôleur) pour organiser le code de manière propre et structurée.

3-2 Les outils utilisé

3-2-1 PhpMyAdmin

PhpMyAdmin est un outil open-source basé sur le web qui permet de gérer les bases de données MySQL. Il offre une interface graphique conviviale pour effectuer des opérations sur des bases de données, telles que la création, la modification et la suppression de tables, la gestion des utilisateurs, l'exécution de requêtes SQL, ainsi que l'importation et l'exportation de données.

3-2-2Daw.io Diagram

Draw.io est un outil en ligne gratuit et open-source pour la création de diagrammes. Il permet de concevoir des diagrammes de flux, des organigrammes, des maquettes, des diagrammes UML

3-2-3 Xampp

XAMPP est un environnement de développement open-source qui permet d'installer facilement Apache, MySQL PHP. Il est principalement utilisé pour développer et tester des applications web localement avant de les déployer sur un serveur en ligne. XAMPP simplifie l'installation de ces outils nécessaires pour exécuter des applications PHP et gérer des bases de données MySQL.

3-3 Architecture de la plateforme

Le Framework laravel se base sur une architecture MVC (Modèle Vues Contrôleur) comme nous indique la figure suivante (figure 3). Le contrôleur est responsable de la logique de contrôle de la plateforme, il sert à gérer les demandes des utilisateurs et à récupérer des données, en tirant parti des modèles, les modèles servent à interagir avec la base de données et récupérer les informations des objets, les vues pour afficher des pages.

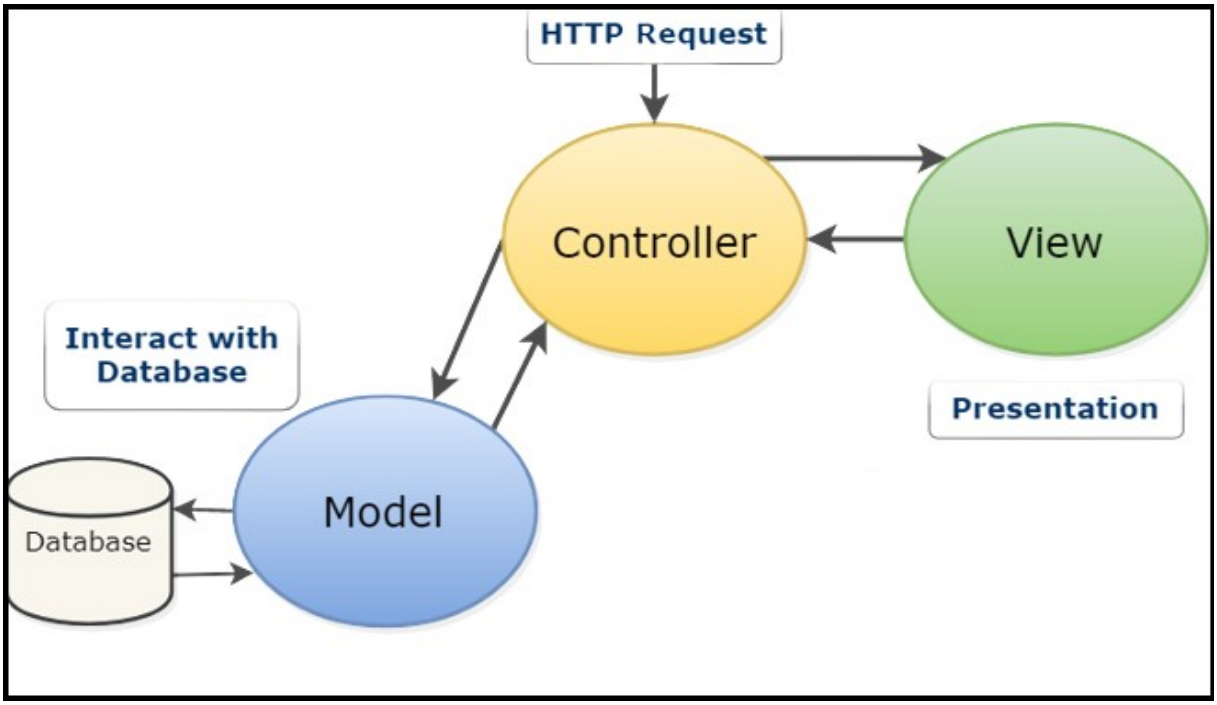

Figure3 : Architecture

3-4 Aperçu visuel de l’interface utilisateur **de la plateforme**

Le maquettage reste une des étapes la plus importantes à définir afin d’avoir un rendu visuel de la plateforme. Cette maquette a été développée dans le but de simplifier l'accès aux diverses fonctionnalités essentielles du système. L’objectif est de garantir une interface claire et moderne, tout en respectant les principes de convivialité et d'accessibilité.

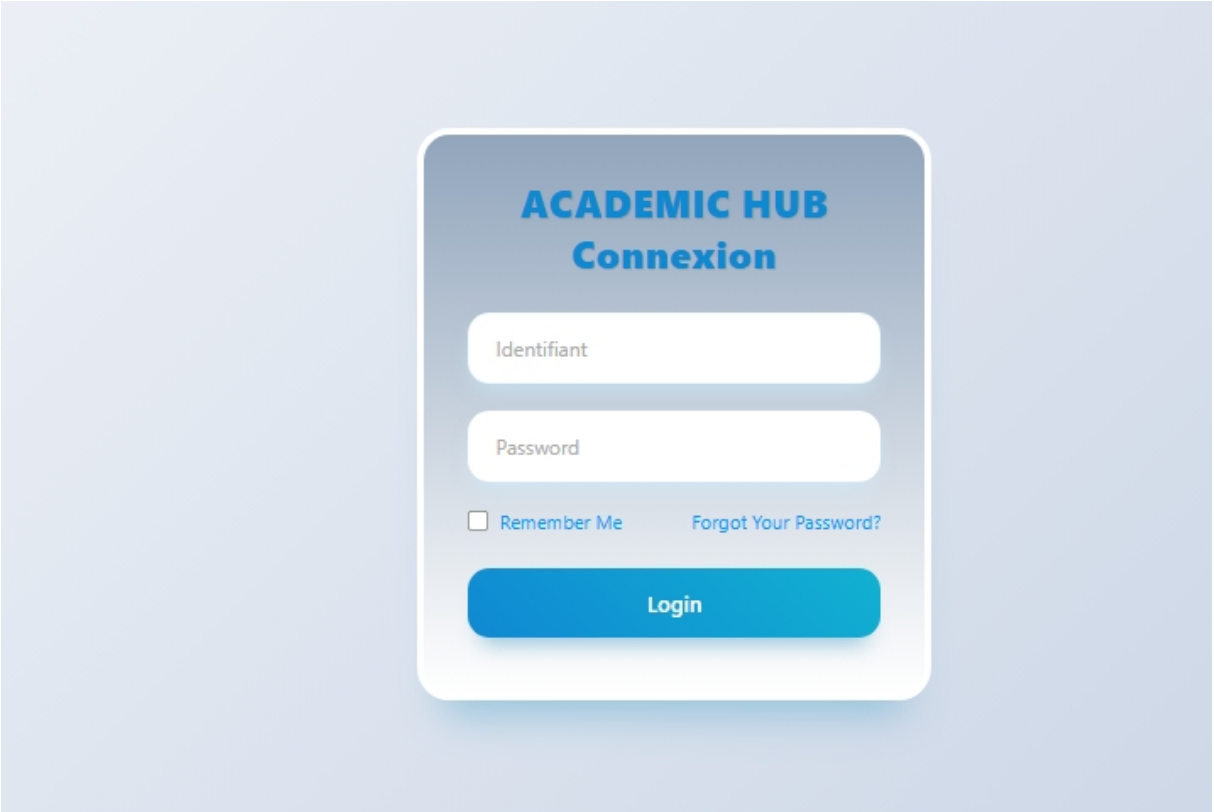

Figure 4 : page de connexion

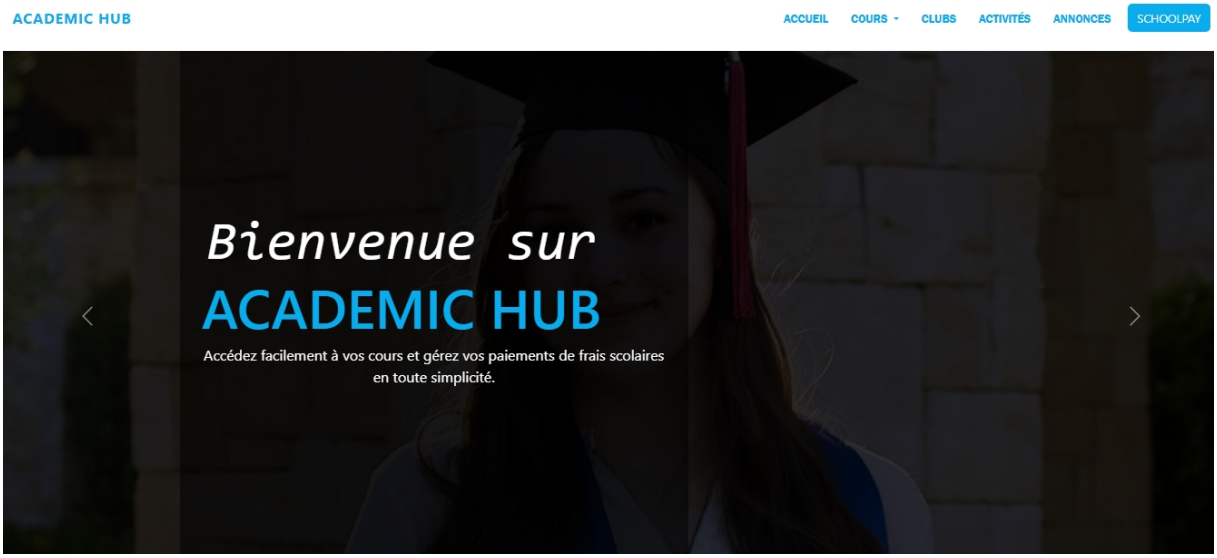

Figure 5 : page d'accueil

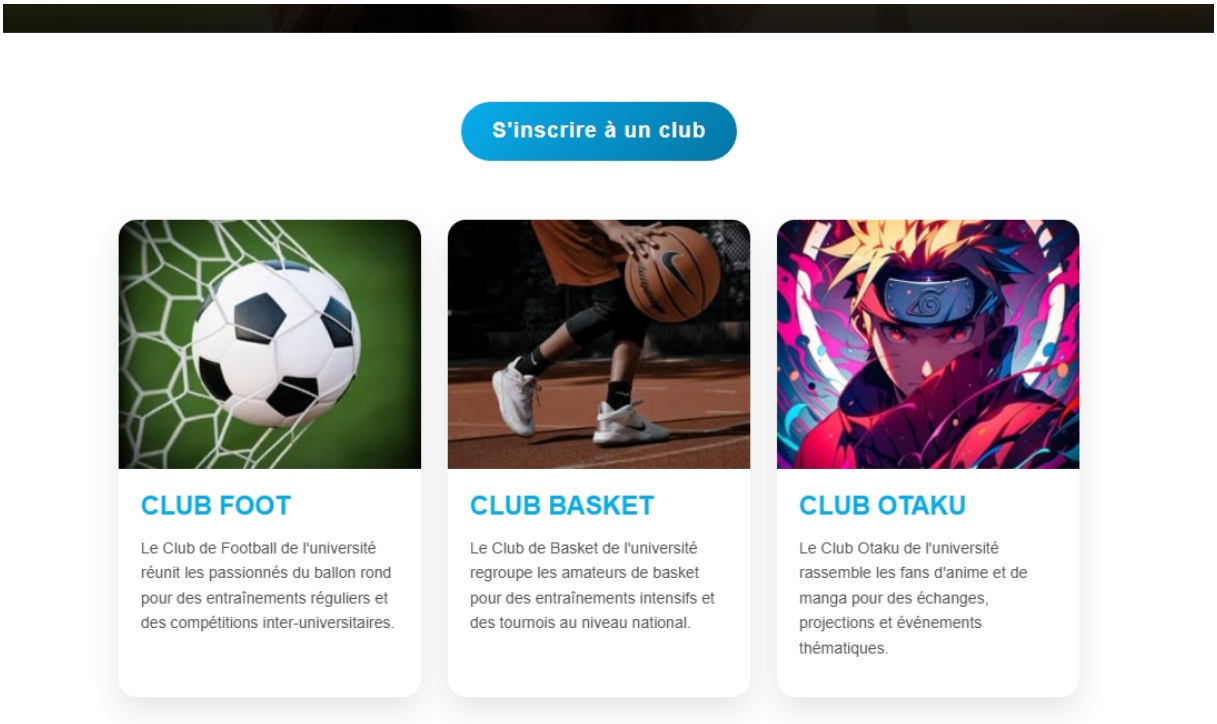

Figure 6 : page Clubs universitaire

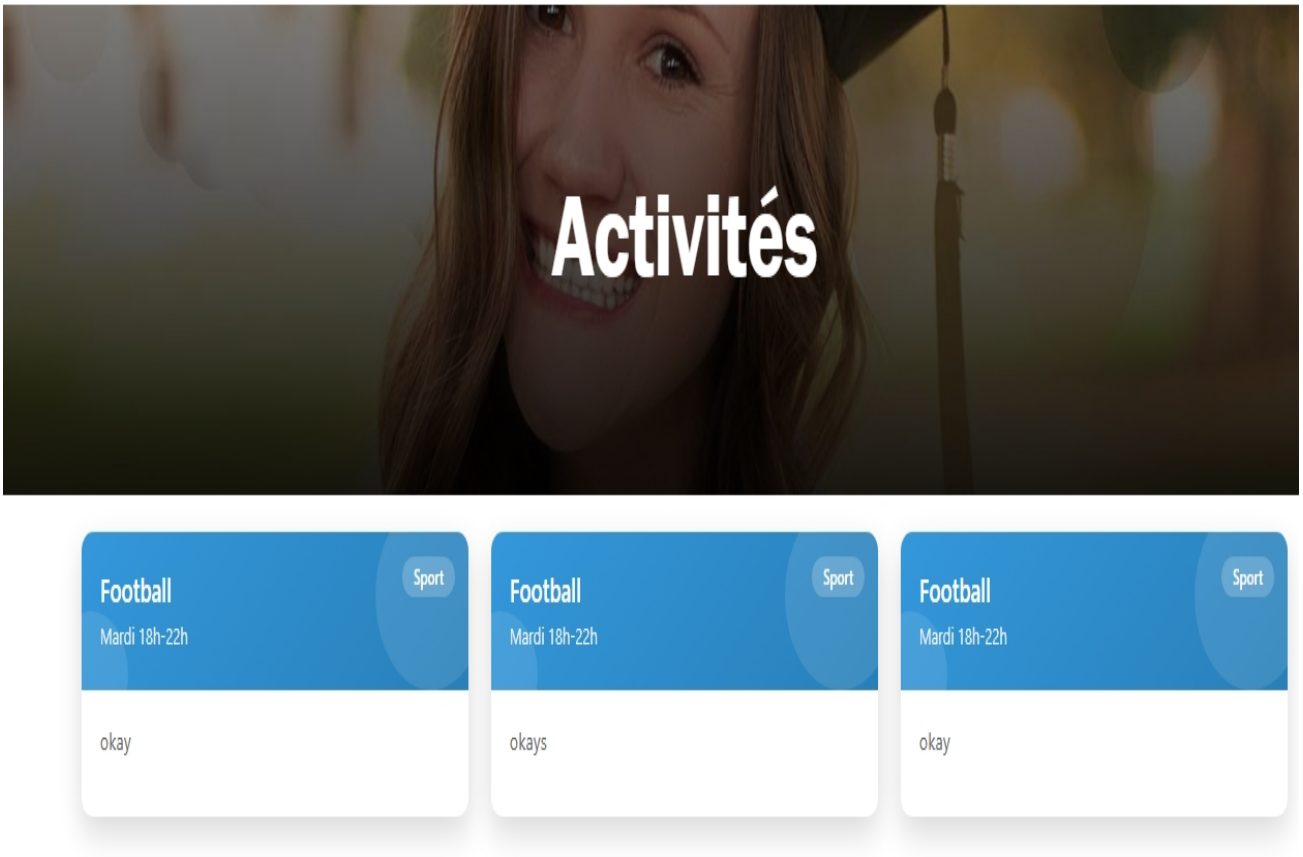

Figure 7 : page activités

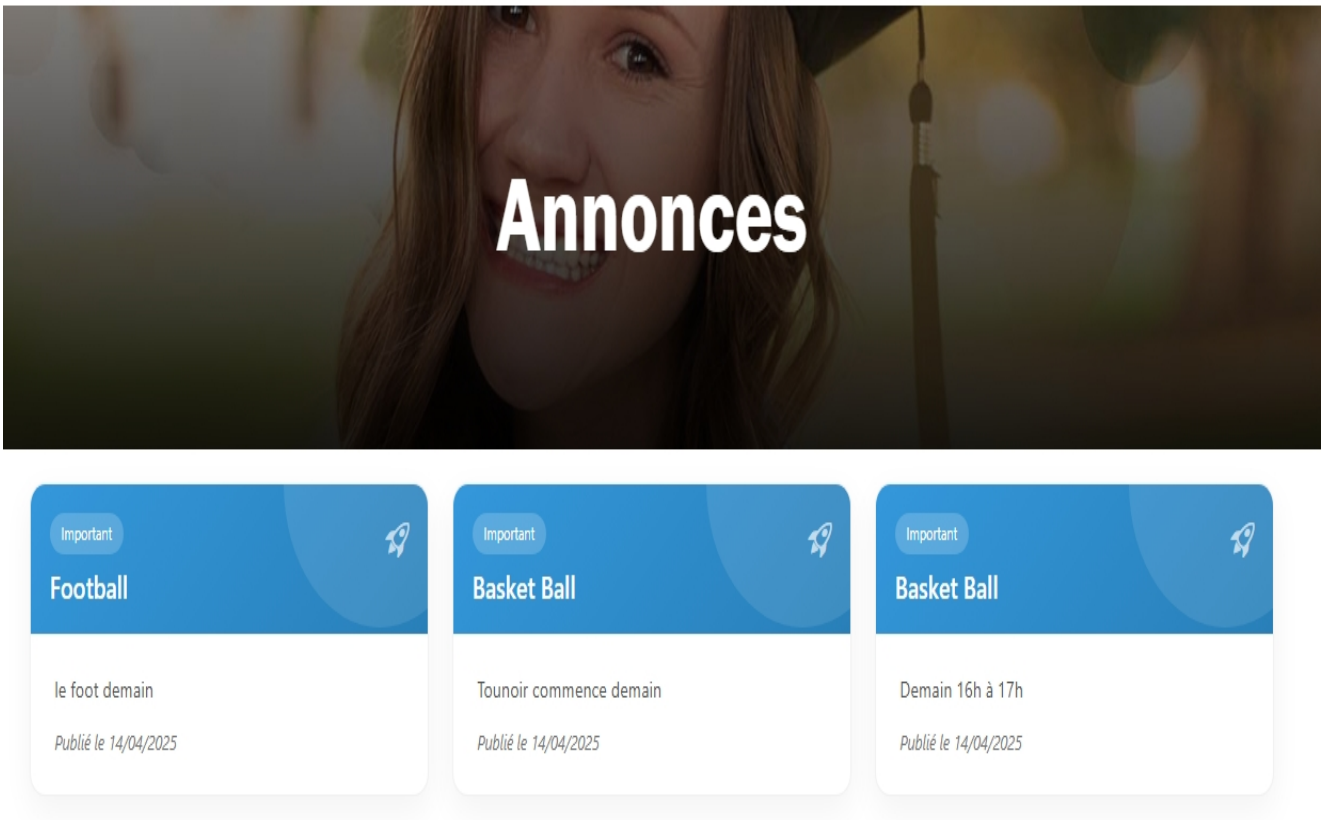

Figure 8 : page annonces

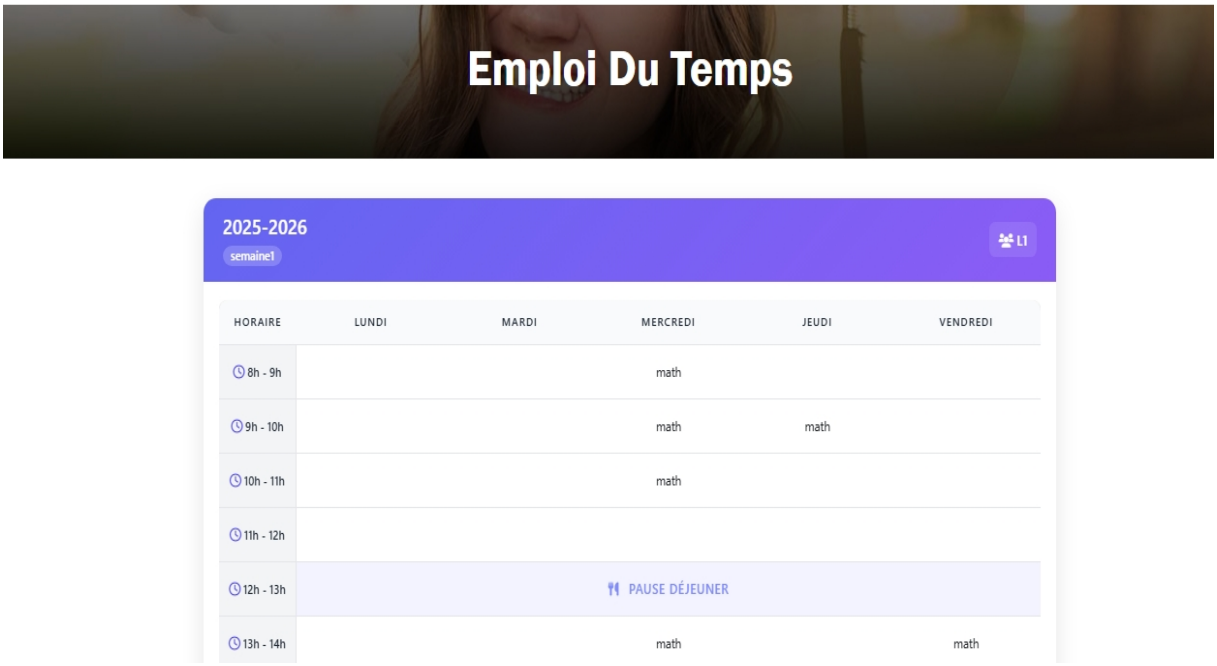

Figure 9 : page emploi du temp

3-5 Implémentation du système

3-5-1 Installation et organisation

Avant de commencer l'installation du Framework laravel, nous devons installer « composer » qui est un gestionnaire de dépendances libre écrit en PHP et permet d'installer les bibliothèques

Pour installer le projet laravel voici la commande **« composer create-project laravel/laravel ACADEMICHUB »**

```
:\Users\Admin>composer create-project laravel/laravel ACADEMICHUB
reating a "laravel/laravel" project at "./ACADEMICHUB"
nstalling laravel/laravel (v12.0.6)
 - Downloading laravel/laravel (v12.0.6)
 - Installing laravel/laravel (v12.0.6): Extracting archive
reated project in C:\Users\Admin\ACADEMICHUB
@php -r "file_exists('.env') || copy('.env.example', '.env');"
```

Figure 10 : Création du projet

Pour lancer localement la plateforme on tape la commande

« php artisan serve » ce qui nous donnera acc é a la page accueil de la plateforme

3-5-2 Les routes

Dans la plateforme les routes constituent un élément fondamental du système, car elles permettent de diriger les différentes requêtes des utilisateurs (étudiants, professeurs et administrateurs) vers les pages

ou les fonctionnalités appropriées, assurant ainsi une navigation fluide et une bonne organisation des accès au sein de l'application.

```php
Route::get('/', function () {
    return view('auth.login');
});

Auth::routes();

Route::get('/etudiantdashboard/header', [App\Http\Controllers\EtudiantDashboardController::class, 'headerpage'])->name('etudiantdashboard.header');
Route::get('/etudiantdashboard/footer', [App\Http\Controllers\EtudiantDashboardController::class, 'footerpage'])->name('etudiantdashboard.footer');

Route::get('/etudiantdashboard/acceuil', [EtudiantDashboardController::class, 'acceuilpage'])->name('etudiantdashboard.acceuil');


Route::get('/etudiantdashboard/club', [App\Http\Controllers\EtudiantDashboardController::class, 'clubpage'])->name('etudiantdashboard.club');

Route::get('/etudiantdashboard/cour', [App\Http\Controllers\EtudiantDashboardController::class, 'courpage'])->name('etudiantdashboard.cour');
Route::get('/etudiantdashboard/cour11', [App\Http\Controllers\EtudiantDashboardController::class, 'courpage11'])->name('etudiantdashboard.cour11');
Route::get('/etudiantdashboard/cour12', [App\Http\Controllers\EtudiantDashboardController::class, 'courpage12'])->name('etudiantdashboard.cour12');
Route::get('/etudiantdashboard/cour13', [App\Http\Controllers\EtudiantDashboardController::class, 'courpage13'])->name('etudiantdashboard.cour13');
Route::get('/etudiantdashboard/annonce', [App\Http\Controllers\EtudiantDashboardController::class, 'annoncepage'])->name('etudiantdashboard.annonce');
Route::get('/etudiantdashboard/activite', [App\Http\Controllers\EtudiantDashboardController::class, 'activitepage'])->name('etudiantdashboard.activite');
Route::get('/etudiantdashboard/schoolpay', [App\Http\Controllers\EtudiantDashboardController::class, 'schoolpaypage'])->name('etudiantdashboard.schoolpay');

Route::get('/etudiantdashboard/inscritclub', [App\Http\Controllers\ClubController::class, 'inscritpage'])->name('etudiantdashboard.inscritclub');
Route::post('/etudiantdashboard/inscritclub', [App\Http\Controllers\ClubController::class, 'clubvalidate'])->name('join-club');


Route::get('/admindashboard/headeradmin', [App\Http\Controllers\AdminDashboardController::class, 'headeradminpage'])->name('dashboard.headeradmin');
```

Figure 11 : Image représentant le fichier de route web.php

3-5-3 Les migrations

Les migrations représentent la partie base de données de la plateforme, elles permettent de mettre à jour le schéma des bases de données. Pour cela nous devons configurer en premier lieu sur le fichier «. env » les champs nécessaires à la connexion à notre base de données créée auparavant. La création de la migration se fait à l’aide de la commande « php artisan make : migration create_nom_table »

```php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('lastname');
            $table->string('firstname');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('password');
            $table->string('login_identifier');
            $table->rememberToken();
            $table->timestamps();
            $table->SoftDeletes();
            $table->boolean('Is_admin')->default(false);
            $table->boolean('Is_etudiant')->default(false);
            $table->boolean('Is_professeur')->default(false);

        });

        Schema::create('password_reset_tokens', function (Blueprint $table) {
            $table->string('email')->primary();
            $table->string('token');
            $table->timestamp('created_at')->nullable();
        });
```

Figure 12 : Image représentant la migration de la table utilisateur

3-5-4 Classe Model

La classe Model joue un rôle central dans Laravel en représentant une table de la base de données. Elle permet d'interagir facilement avec les données grâce à l'ORM Eloquent, qui facilite les opérations de lecture, d'écriture, de mise à jour et de suppression. La création du model se fait à l'aide de la commande « php artisan make : model NomModel »

```php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Cour extends Model
{
    protected $fillable = [
        'nom_cour',
        'categorie',
        'duree',
        'user_id'
    ];



    public function user()
    {
        return $this->belongsTo(User::class);
    }

    public function document()
    {
        return $this->hasMany(Document::class);
    }


}
```

Figure 13 : Image représentant la classe modèle

3-5-5 Les contrôleurs

Le contrôleur joue un rôle intermédiaire entre les modèles et les vues dans une application Laravel ; il traite les requêtes HTTP, exécute la logique métier et renvoie les réponses appropriées à l'utilisateur. La création du contrôleur se fait à l'aide de la commande « php artisan make : Controller NomController »

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Document;
use App\Models\Cour;

class professeurDashboardController extends Controller
{



    public function headerpage(){

        return view('professeurdashboard.header');

       }


    public function indexpagepage(){

        return view('professeurdashboard.index');

       }



    public function courcreatepage(){

     return view('professeurdashboard.courcreate');

    }
```

Figure 14 : Image représentant le Contrôleur

3-5-6 Les vues

Le processus de visualisation des pages se fait à travers des vues ainsi que les routes.

```
div class="login-container">
   <div class="heading">ACADEMIC HUB<br>Connexion</div>
   <div class="form">
       <form action="{{route('login')}}" method="POST">
           <div class="input-group">
               <input type="text" id="Identifiant" placeholder="Identifiant"
                   class="form-control @error('login_identifier') is-invalid @enderror"
                   name="login_identifier" value="{{ old('login_identifier') }}"
                   required autocomplete="email" autofocus>

               @error('login_identifier')
               <span class="invalid-feedback" role="alert">
                   <strong>{{ $message }}</strong>
               </span>
               @enderror
           </div>

           <div class="input-group">
               <input type="password" id="password" placeholder="Password"
                   class="form-control @error('password') is-invalid @enderror"
                   name="password" required autocomplete="current-password">

               @error('password')
               <span class="invalid-feedback" role="alert">
                   <strong>{{ $message }}</strong>
               </span>
               @enderror
           </div>

           <div class="options-row">
               <div class="remember-me">
                   <input type="checkbox" name="remember" id="remember" {{ old('remember') ? 'checked' : '' }}>
                   <label for="remember">{{    ('Remember Me') }}</label>
```

Figure 15 : Image représentant la vue de connexion

Conclusion

Le développement de cette plateforme de gestion des cours, des activités universitaires et de paiement de frais de scolarité répond à un besoin réel de modernisation et d'optimisation des processus académiques. Grâce à une interface conviviale et des fonctionnalités adaptées aux différents acteurs (étudiants, professeurs et administrateurs), cette solution numérique facilite la communication, centralise les informations essentielles et automatise plusieurs tâches administratives. Elle permet non seulement un gain de temps considérable, mais aussi une meilleure organisation au sein de l'établissement. Ce projet s'inscrit pleinement dans une démarche de digitalisation de l'environnement universitaire, tout en garantissant sécurité**,** accessibilité et efficacité**.**